

课程笔记

Gemini 2.5 Pro + Agent = IMO 金牌： Actor-Critic Workflow

基于公开课程资料整理

2025

Gemini 2.5 Pro + Agent = IMO 金牌

Gemini 2.5 Pro Capable of Winning Gold at IMO 2025*

Yichen Huang[†] Lin F. Yang[‡]

July 2025

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 25 分钟

视频链接：<https://www.bilibili.com/video/BV1Aq8bz1Emj>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 工作背景与核心发现	2
1.1 令人振奋的结果	2
1.2 本章小结	2
2 完整 Workflow 流程	2
2.1 六步流程	2
2.2 Evaluator-Optimizer 模式	2
2.3 初始解质量的决定性作用	3
2.4 本章小结	3
3 Step 1 Prompt 分析: 生成初始解	3
3.1 Prompt 结构	3
3.2 输出格式的设计意图	3
3.3 Step 2: Self-Improvement	3
3.4 本章小结	4
4 Verification Prompt 分析	4
4.1 Verifier 的角色与指令	4
4.2 本章小结	4
5 Correction 与迭代循环	4
5.1 Correction 环节	4
5.2 完整迭代流程	5
5.3 本章小结	5
6 Gemini API 使用注意事项	5
7 总结与延伸	5

1 工作背景与核心发现

1.1 令人振奋的结果

本期介绍了一篇对作者启发极大的工作：**原生 Gemini 2.5 Pro**（非 DeepMind 官方 tune 过的版本）通过精心设计的 Agent Workflow，求解了 IMO 2025 六道题中的前五道，达到人类金牌水平。

核心发现

关键词是 **capable**——模型本身有能力解决这些极其困难的数学问题，此前表现不佳是因为没有通过合适的 Workflow 来最大化激发模型的潜力。这对 Agent 设计的启示是巨大的。

1.2 本章小结

这一工作证明了 Agent Workflow 设计的重要性：同样的模型，不同的 Workflow 可以带来质的飞跃。

2 完整 Workflow 流程

2.1 六步流程

1. **Step 1: 初始求解**——基于 Prompt 和 Gemini 2.5 Pro 产生一个初始解
2. **Step 2: Self-Improvement**——共享 Step 1 的上下文，继续 refine（因为 Step 1 可能因 Thinking Budget 耗尽而仓促结束）
3. **Step 3: Verification**——对 Step 2 的解进行验证，输出 verdict（有效/无效）和 bug report
4. **Step 4: Bug Report**——提取具体的错误位置和原因
5. **Step 5: Correction**——基于 bug report 修正 solution
6. **Step 6: 循环**——Step 3-5 不断循环

终止条件

- **接受**：某个 solution 连续 **5 次**通过 verification → 认为解决了问题
- **拒绝**：循环 **10 次**仍未通过 → 放弃当前方向，重新 sample

2.2 Evaluator-Optimizer 模式

与 Building Effective Agents 中的经典模式对应

这个 Workflow 本质是 **Evaluator-Optimizer** 模式：

- Generator（生成器）产生 solution
- Evaluator（评估器）验证并给出 feedback
- 接受则输出，拒绝则基于 feedback 重新生成

2.3 初始解质量的决定性作用

成功的关键在于初始解

如果 Step 2 生成的初始解与正确解有较大的 overlap，那么经过后续的验证-修正循环大概率能最终 solve 问题。但如果初始解的方向就是错的（偏差太远），后续的纠正大概率救不回来。

2.4 本章小结

整个 Workflow 是一个标准的 Generate-Verify-Correct 迭代循环，终止条件基于连续通过/失败次数的计数。

3 Step 1 Prompt 分析：生成初始解

3.1 Prompt 结构

Step 1 的 System Prompt 是一个标准的结构化 Markdown 文本，分为三部分：

1. 核心指令：

- 严谨性至关重要（IMO 大部分是证明题，最终判据是逻辑严谨性）
- 诚实对待结果——不要硬凹（避免 Reward Hacking）
- 全文使用 LaTeX

2. 输出格式（Adaptive Prompt）：

- Summary：包含 Verdict（是否找到完整方案）和方法概述
- Detailed Solution：详细的解决方案

3. Self-Verification 指令

3.2 输出格式的设计意图

Verdict 用于分支判断

Summary 中的 Verdict 不仅是信息展示，更是后续 Workflow 的**分支判断依据**——如果 Verdict 为 false（未找到完整解），则跳过后续验证，直接重新 sample。Detailed Solution 则送入 Verification 环节。

本文代码使用**正则表达式**提取这些结构化输出（而非结构化输出 API），因为 Gemini 2.5 Pro 的指令遵循能力足够强。

3.3 Step 2: Self-Improvement

Step 2 与 Step 1 **共享上下文**。其 Prompt 很简单：“你有一个机会改进你的结果——仔细 review 你的 solution，纠正错误，填充 justification gap。”

引入 Step 2 的原因是 Step 1 中 Thinking Budget（32768 tokens）可能被打满，模型被迫仓促结束思考。Step 2 给予模型继续思考的机会。

3.4 本章小结

Prompt 设计的精髓在于：结构化输出为后续 Workflow 的分支判断和数据流向提供了基础。

4 Verification Prompt 分析

4.1 Verifier 的角色与指令

Verification 的 System Prompt 同样是结构化 Markdown，包含：

1. **角色扮演**：你是卓越的数学家，IMO 级别严谨的审阅人
2. **核心说明**：你仅仅是 **Verifier**，不是 **Solver**——不要尝试纠正错误或填充 gap，只需 step by step 验证
3. **问题分类**：
 - **Critical Error**：严重逻辑错误（如 $A > B, C > D$ 推出 $A - C > B - D$ ）
 - **Gap**：跳步、逻辑断裂
4. **输出格式**：Summary（含 Verdict：valid/invalid）+ List of Findings（bug report）+ Detailed Verification Log

Verifier 不应尝试解题

Verification Prompt 明确要求 Verifier 不要尝试纠正错误或填充 gap，只做验证。这种**角色分离**是 Workflow 设计的关键——Generator 和 Verifier 各司其职。

4.2 本章小结

Verification Prompt 通过严格的角色定义和结构化输出格式，确保 Verifier 只做验证、不做求解，其输出的 Verdict 和 Bug Report 驱动后续的 Correction 和循环终止判断。

5 Correction 与迭代循环

5.1 Correction 环节

当 Verifier 判定 solution 无效时，提取 bug report 送入 Correction 环节：

- 输入：原始问题 + 当前 solution + bug report
- Prompt：“Below is a bug report. If you agree with it, fix the issues. Your output should follow the system prompt instructions.”
- 输出：修正后的新 solution

5.2 完整迭代流程

以第一道题为例：

1. 初始探索：Step 1 + Step 2 生成初始解
2. 验证：Verifier 判定 No (有 bug)
3. 纠正：基于 bug report 生成新解 → 验证 No → 纠正 → 验证 No
4. 第 4 次纠正后通过验证：correct count = 1
5. 连续验证通过 5 次，最终接受
6. 共经历 8 次迭代，外部循环仅 1 次 run

5.3 本章小结

整个迭代循环展示了 Actor-Critic 模式的强大：即使初始解有多处错误，通过反复的验证-纠正循环，最终可以收敛到正确解。

6 Gemini API 使用注意事项

技术细节

- Gemini API 的角色是 system / user / model (而非 system / user / assistant)
- Thinking 输出是 summarize 后的结果，不是原生的 thinking 过程
- Thinking Budget 设置为 32768 tokens，但实际有时会达到 34000+ (控制不严格)
- 由于 Thinking 过程很长 (数万 token)，使用 HTTP Request 容易超时，建议使用 SDK + Streaming

7 总结与延伸

1. **Generate-Verify-Correct 循环**是一个强大且通用的 Workflow 模式，本文在 IMO 问题上的成功验证了其有效性。
2. **初始解质量决定成败**：如果初始解方向正确 (与标准解有较大 overlap)，后续循环大概率能修正细节；如果方向错误，则难以纠正。
3. **Prompt 设计是 Workflow 的灵魂**：结构化的输出格式为自动化的分支判断和数据流转提供了基础。
4. **角色分离**：Generator 只管生成，Verifier 只管验证，Corrector 只管修正——各司其职是 Multi-Agent 设计的核心原则。
5. **Agent 设计的价值**：同样的模型 (Gemini 2.5 Pro)，通过 Workflow 设计可以从”做不出”变为”金牌水平”。